OnboardAI Developer Guide

Prepared for the Developer role. Rohith K Nair, DL.AI.U4AID25043.

This guide explains how the system works internally. It covers the logic behind every module, the mathematics of the models, the flow of a request from browser to database and back, and the reasoning behind the design choices that are not obvious from reading the code alone.

The code excerpts below are extracted automatically from the source files when this guide is built, so they always match the implementation.

---

1. Running the Project

1.1 First time setup

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

1.2 Build the data, the models and the reports

Order matters. The reports read the trained models, and the demo documents are needed by the test suite.

```
python seed.py                 # database with six employees across two organisations
python -m ml.knowledge_seed     # policy corpus for retrieval
python -m ml.make_demo_docs     # seven PDFs named by expected verdict
python -m ml.train_classifier   # document type classifier
python -m ml.train_nlp          # intent and sentiment
python -m ml.train_risk         # stall risk with ablations
python -m ml.data_report        # dataset card, quality checks, split audit
python -m ml.eda                # ten figures and observations
python -m pytest tests -q       # 55 checks
```

1.3 Run the application

Two terminals, backend first.

```
cd backend && venv\Scripts\activate && uvicorn main:app --port 8000
npm install && npm run dev
```

---

2. Architecture

2.1 The three layers

The system is a normal web application with a machine learning layer wired into the request path rather than bolted on beside it.

```
React 19 + Vite + Tailwind          browser, two role based views
            |
            | fetch, JSON over HTTP
            v
  FastAPI + SQLAlchemy + SQLite       routes, ORM, event stream
            |
            +-- ml/        document pipeline, NLP, retrieval, risk scoring
            +-- agents/    six specialists over a local language model
```

2.2 The three data streams

Every capability belongs to one of three streams, and each stream has its own dataset, its own generator and its own model.

Stream	Input	Model	Output
Documents	Uploaded PDF or image	Calibrated LinearSVC over TF-IDF	Document type and verdict
Messages	Chat text	Two logistic regressions	Intent and sentiment
Journeys	Database and event stream	Logistic regression	Stall probability

2.3 The event stream, and why it exists

There are two audit tables and they are not duplicates of one another.

`audit_logs` holds prose for the human timeline in the HR interface. `events` holds structured records with a type, an actor, a JSON payload and a confidence. The risk model reads features only from `events`.

The consequence is important for anybody adding a feature. If an action does not write an event, the machine learning layer cannot see it. Writing to `audit_logs` alone makes an action visible to a person and invisible to a model.

```
def record(
    db,
    event_type: str,
    actor: str,
    emp_id: str | None = None,
    payload: dict | None = None,
    confidence: float | None = None,
    audit: str | None = None,
    commit: bool = True,
):
    event = models.Event(
        empId=emp_id,
        event_type=event_type,
        actor=actor,
        payload=payload or {},
        confidence=confidence,
    )
    db.add(event)

    if audit and emp_id:
        db.add(models.AuditLog(time="Just now", action=audit, empId=emp_id))

    if commit:
        db.commit()
    return event
```

---

3. Repository Map

```
backend/
  main.py              app, CORS, /api/health
  database.py          engine and session, path anchored to the file
  models.py            eight tables
  schemas.py           pydantic response models
  seed.py              fixture data, dates relative to today
  llm_provider.py      the only module that names a model vendor
  storage.py           upload storage with magic byte verification
  events.py            the single writer to the event stream

  ml/
    synth.py           document corpus generator
    text_synth.py      message corpus generator
    risk_synth.py      journey dataset generator
    data_report.py     dataset card, quality audit, split audit
    eda.py             ten figures with generated observations
    train_classifier.py
    train_nlp.py
    train_risk.py
    extract.py         PDF text layer, OCR fallback
    classifier.py      document classifier inference
    fields.py          date, name and identifier extraction
    validate.py        validation rules with severities
    pipeline.py        extract to verdict
    nlp_models.py      intent and sentiment inference
    pii.py             redaction before text leaves the process
    rag.py             hybrid retrieval with access control
    risk_model.py      feature building, scoring, attribution

  agents/
    specialists.py     six agents, their tools and their prompts
    chatbot.py         routing and dispatch

  routes/              api.py, documents.py, chat.py
  tests/               34 cases, 55 checks
```

---

4. The Data Generators

All three datasets are generated. The design principle throughout is that the generated task must be hard enough for the resulting metric to mean something.

4.1 Document corpus

`ml/synth.py` holds 21 templates across 9 classes. Each sample fills the template slots from pools of names, dates and identifiers, then degrades the result.

Degradation runs in three tiers. The mix is roughly 40 per cent clean, 38 per cent degraded and 22 per cent severe.

```
def _degrade(body, tier, rng, slots):
    """Apply tier-appropriate damage to a clean document."""
    if tier == "clean":
        if rng.random() < 0.30:
            body = _add_noise(body, rng, rate=rng.uniform(0.005, 0.02))
        if rng.random() < 0.15:
            body = body.upper()
        return body

    if tier == "degraded":
        if rng.random() < 0.55:
            body = _strip_header(body, rng)
        if rng.random() < 0.45:
            body += "\n" + rng.choice(_CROSS_CLASS_NOISE).format(**slots)
        body = _add_noise(body, rng, rate=rng.uniform(0.03, 0.09))
        if rng.random() < 0.25:
            body = body.upper()
        return body

    # severe: a bad phone photo of a crumpled page
    body = _strip_header(body, rng)
    body = _fragment(body, rng)
    for _ in range(rng.randint(1, 2)):
        body += "\n" + rng.choice(_CROSS_CLASS_NOISE).format(**slots)
    body = _add_noise(body, rng, rate=rng.uniform(0.10, 0.20))
    if rng.random() < 0.35:
        body = body.upper()
    return body
```

The noise function models OCR damage using confusions a real OCR engine actually makes, such as reading zero as capital O, or the pair `rn` as `m`.

```
def _add_noise(text, rng, rate=0.02):
    """Approximate OCR damage so the classifier cannot rely on clean spelling."""
    chars = list(text)
    for _ in range(int(len(chars) * rate)):
        i = rng.randrange(len(chars))
        roll = rng.random()
        if roll < 0.5:
            src, dst = rng.choice(_OCR_CONFUSIONS)
            if chars[i] == src:
                chars[i] = dst
        elif roll < 0.75:
            chars[i] = ""
        else:
            chars[i] = chars[i].swapcase()
    return "".join(chars)
```

The reason all of this exists is worth understanding before changing it. The first version of the generator had no degradation, and the classifier trained on it scored 1.000 on every metric with zero cross validation

variance. That is not a good model, it is a broken benchmark. The classes were separable on vocabulary alone.

4.2 Message corpus

`ml/text_synth.py` produces one corpus carrying two independent labels, intent and sentiment.

Two mechanisms exist specifically to stop the problem being trivially easy.

The first is a pool of vague messages. These are assigned the intent the person actually had, but the text does not reveal it. This produces error that no model can remove, which is correct, because the same situation arises with real messages.

The second blends a clause from a different intent into about a quarter of messages, while the label stays with the dominant intent.

```
_VAGUE = [
    "any update?", "still waiting", "hello?", "can someone help",
    "what's happening with this", "following up", "please advise",
    "checking in again", "?", "sorry to chase", "is anyone there",
]

# Clauses that drag a message toward a second intent.
_BLEND_CLAUSES = [
    "also I still can't log in to {system}",
    "and I've not had my {doc} confirmed",
    "by the way when is the {topic} session",
    "also what's the leave policy",
    "and when does my salary come through",
    "also what stage am I at",
]


def _fill(template, rng):
```

There is a subtlety in the sentiment side that caused a real defect. Both the positive and the neutral prefix pools contain an empty string, which meant a message could be labelled positive while containing no positive signal at all. That is mislabelled training data. The current code guarantees a marker unless the intent carries the tone by itself.

```
            prefix = rng.choice(_SENTIMENT_PREFIX[sentiment])
            suffix = rng.choice(_SENTIMENT_SUFFIX[sentiment])

            # Escalation bodies are inherently angry and smalltalk bodies
            # inherently warm, so those carry their tone without any marker.
            implicit = intent in ("escalation", "smalltalk")

            if sentiment != "neutral" and not implicit and not prefix and not suffix:
                # Both pools contain an empty option, so without this a positive
                # or negative example could end up with no sentiment marker at
                # all: text indistinguishable from neutral, carrying another label.
                if rng.random() < 0.5:
                    prefix = rng.choice([p for p in _SENTIMENT_PREFIX[sentiment] if p])
                else:
                    suffix = rng.choice([s for s in _SENTIMENT_SUFFIX[sentiment] if s])

            # Mixed signals: polite opener, unhappy substance. The label follows
            # the substance, which is what makes these genuinely hard.
            if sentiment == "negative" and rng.random() < 0.14:
                prefix = rng.choice([
                    "Thanks for getting back to me, but ",
                    "I appreciate the help, however ",
                    "Sorry to be a pain, but ",
                    "No offence to anyone, but ",
```

4.3 Journey dataset

`ml/risk_synth.py` is the most interesting generator and the one most likely to be asked about. It is a latent variable process rather than independent sampling.

Three hidden traits are drawn per simulated employee. Every observable feature is then derived as a noisy measurement of those traits.

```
engagement ~ N(0, 1)        how proactively the hire works through onboarding
friction   ~ N(0, 1) + dept  how much the process obstructs them
support    ~ N(0, 1)        quality of manager, buddy and HR contact
```

For example, inactivity is exponential with a rate set by engagement and friction, and sentiment is a noisy hyperbolic tangent of a weighted combination of all three.

```
days_since_last_activity ~ Exponential(exp(0.75 - 0.65*engagement + 0.30*friction))
avg_sentiment            = tanh(0.55*engagement - 0.85*friction + 0.45*support + e)
pct_tasks_complete       = sigmoid(1.1*engagement - 0.5*friction + e) * (0.35 + 0.75*elapsed)
```

The outcome combines the same traits with two direct blocking effects and, critically, a term that no feature observes.

```
    unobserved = rng.normal(0, 1, n)
    stall_logit = (
        -0.75
        + 1.10 * friction
        - 1.05 * engagement
        - 0.55 * support
        + 0.85 * unobserved
        + 0.35 * is_remote
        - 0.20 * seniority
        # Direct blocking effects, not mediated by the latents: a rejected
        # document and a silent hire each stall onboarding on their own.
        + 0.42 * np.minimum(docs_rejected, 3)
        + 0.055 * days_since_last_activity
    )
    stalled = rng.binomial(1, _sigmoid(stall_logit))
```

That unobserved term places a ceiling on achievable performance by construction. If a model ever scored near 1.0 on this data it would mean the generator had leaked the label, not that the model was good.

Two features, `response_latency_hours` and `is_remote`, were removed from the feature list during the project. They are still simulated and still influence the outcome, but are no longer offered as inputs. The reason is in section 9.

---

5. Preprocessing

5.1 Text vectorisation

The feature representation is a union of two TF-IDF vectorisers.

```
def build_features():
    """Word n-grams carry topic; char n-grams survive OCR damage that breaks words."""
    return FeatureUnion([
        ("word", TfidfVectorizer(
            analyzer="word", ngram_range=(1, 2), min_df=2,
            sublinear_tf=True, strip_accents="unicode", lowercase=True,
        )),
        ("char", TfidfVectorizer(
            analyzer="char_wb", ngram_range=(3, 5), min_df=3,
            sublinear_tf=True, lowercase=True, max_features=60000,
        )),
    ])
```

TF-IDF weights a term by how often it appears in a document against how rare it is across the corpus. With `sublinear_tf` the term frequency becomes `1 + log(tf)` rather than `tf`, which stops a word repeated twenty times in one document from dominating its vector.

`min_df=2` drops any term appearing in only one document. Such a term cannot generalise, it can only memorise.

The character n-grams are the deliberate design choice. Word features are destroyed by the character level noise in the severe tier, because a corrupted word becomes a different token entirely. Character n-grams of length three to five survive, because most of the substrings in a lightly corrupted word are still intact. This is why the severe tier scores 0.937 rather than collapsing.

`char_wb` restricts the character windows to within word boundaries, which avoids generating n-grams that straddle two unrelated words.

5.2 Feature scaling

The tabular features span different units, from a percentage to a day count to a bounded sentiment score. Logistic regression coefficients are only comparable if the inputs are on the same scale, so each column is standardised to mean zero and unit variance.

The scaler sits inside a `Pipeline`, which matters for correctness rather than tidiness. During cross validation the pipeline refits the scaler on each training fold alone, so the mean and standard deviation of the test fold never influence the transformation.

---

6. Splitting and Leakage

Every dataset uses a 75 and 25 stratified split with `random_state=42`.

Stratification holds each class proportion constant across both folds. Without it, a small class can be unlucky and end up underrepresented in the test fold, which makes its reported score unstable.

The split is audited rather than assumed correct.

```
def split_audit(texts, labels):
    """Verify stratification held and that no document leaked across the split."""
    X_tr, X_te, y_tr, y_te = train_test_split(
        texts, labels, test_size=TEST_SIZE, random_state=SEED, stratify=labels)

    overall = Counter(labels)
    tr, te = Counter(y_tr), Counter(y_te)
    drift = {c: abs(tr[c] / len(y_tr) - overall[c] / len(labels)) for c in overall}

    return {
        "train_size": len(X_tr),
        "test_size": len(X_te),
        "split_ratio": f"{100 * (1 - TEST_SIZE):.0f}/{100 * TEST_SIZE:.0f}",
        "random_state": SEED,
        "stratified": True,
        "max_class_proportion_drift": max(drift.values()),
        "train_class_counts": dict(sorted(tr.items())),
        "test_class_counts": dict(sorted(te.items())),
        "overlap_documents": len(set(X_tr) & set(X_te)),
    }
```

The audit found that the document corpus is clean but the message corpus is not. Short messages such as a plain thank you are produced identically by different random draws, so some appear in both folds.

The obvious hypothesis is that this inflates the scores. That was tested rather than assumed.

```
def leakage_impact(texts, labels):
    """Quantify memorisation: score the leaked and clean halves of the test set apart.

    One training run. If the model scores materially higher on test strings that
    also appear in training, that gap is memorisation rather than generalisation.
    """
    from ml.train_nlp import build_pipeline

    X_tr, X_te, y_tr, y_te = train_test_split(
        texts, labels, test_size=TEST_SIZE, random_state=SEED, stratify=labels)

    train_set = set(X_tr)
    leaked = [i for i, t in enumerate(X_te) if t in train_set]
    clean = [i for i, t in enumerate(X_te) if t not in train_set]

    pipe = build_pipeline()
    pipe.fit(X_tr, y_tr)
    preds = pipe.predict(X_te)

    def acc(idx):
        if not idx:
            return None
        return float(np.mean([preds[i] == y_te[i] for i in idx]))

    overall, leaked_acc, clean_acc = acc(range(len(y_te))), acc(leaked), acc(clean)
    return {
        "test_size": len(y_te),
        "leaked_rows": len(leaked),
        "clean_rows": len(clean),
        "accuracy_overall": overall,
        "accuracy_on_leaked": leaked_acc,
        "accuracy_on_clean": clean_acc,
        "memorisation_gap": (leaked_acc - clean_acc)
        if (leaked_acc is not None and clean_acc is not None) else None,
    }
```

The result contradicted the hypothesis. Accuracy on the leaked rows came out lower than on the clean rows, because the duplicated messages are the short ambiguous ones, which are the hardest to classify in the first place. Duplication is concentrated in the difficult cases, so it depresses the leaked score rather than inflating it.

The leakage remains a defect. Two tests assert its presence and bound its size so that it cannot grow unnoticed.

---

7. The Document Pipeline

7.1 Stages

```
extract text  ->  classify type  ->  extract fields  ->  validate  ->  decide
```

7.2 Text extraction

Most onboarding documents are digital PDFs with a real text layer, which is both faster and far more accurate than OCR. OCR is therefore a fallback, and an optional dependency. With no OCR engine installed the pipeline still runs and reports honestly that it could not read the image.

```
def extract_text(path: Path, content_type: str) -> dict:
    """Return {text, method, char_count}. Never raises on unreadable input."""
    path = Path(path)

    try:
        if content_type == "application/pdf":
            text = _pdf_text(path)
            if len(text) >= MIN_TEXT_CHARS:
                return {"text": text, "method": "pdf:text-layer", "char_count": len(text)}
            scanned, method = _ocr(path)
            if scanned:
                return {"text": scanned, "method": f"pdf:{method}", "char_count": len(scanned)}
            return {
                "text": text,
                "method": "pdf:no-text-layer",
                "char_count": len(text),
                "note": "PDF appears to be a scan and no OCR engine is installed.",
            }

        text, method = _ocr(path)
        note = None if text else "No OCR engine installed; install pytesseract or easyocr."
        return {"text": text, "method": method, "char_count": len(text), "note": note}

    except Exception as exc:
        logger.exception("Text extraction failed for %s", path)
        return {"text": "", "method": "error", "char_count": 0, "note": f"{type(exc).__name__}: {exc}"}
```

7.3 Field extraction, and the date problem

The date logic is the part most worth reading, because an earlier version contained a real and damaging bug.

Identity documents routinely print the issue date and the expiry date on the same line. The first implementation searched the whole line for keywords, found the word expiry somewhere on it, and assigned the first date to that role. The result was that a valid passport was marked as expired.

The current implementation assigns each date the role of the label nearest to its left.

```
def _nearest_role(segment: str) -> str | None:
    """Role of the label closest to the end of `segment`, if any.

    Identity documents routinely put issue and expiry on one line, so a date
    must take the role of the label immediately to its left, not of any label
    appearing somewhere on the line.
    """
    best_role, best_index = None, -1
    for role, hints in _ROLE_HINTS:
        for hint in hints:
            index = segment.rfind(hint)
            if index > best_index:
                best_role, best_index = role, index
    return best_role
```

```
def _classify_dates(text: str) -> dict:
    found = {"issue": None, "expiry": None, "dob": None}
    all_dates = []
    pending_role = None

    for line in text.split("\n"):
        lowered = line.lower()
        matches = list(_DATE_RE.finditer(line))

        if not matches:
            # A label on its own line applies to dates on the line below it.
            pending_role = _nearest_role(lowered)
            continue

        for match in matches:
            parsed = _parse(match.group(0))
            if not parsed:
                continue
            iso = parsed.isoformat()
            all_dates.append(iso)

            role = _nearest_role(lowered[: match.start()]) or pending_role
            if role and not found[role]:
                found[role] = iso

        pending_role = None

    return {"dates": found, "all_dates": sorted(set(all_dates))}
```

A second fault in the same module: the abbreviation `cert` matched the opening of the word CERTIFICATE in a document title and captured `IFICATE` as the identifier. The pattern now requires a qualifier token to follow.

7.4 The decision policy

```
def decide(checks: list[dict], confidence: float | None) -> tuple[str, str]:
    """Return (verdict, human-readable reason)."""
    blocking = [c for c in checks if not c["passed"] and c["severity"] == "blocking"]
    if blocking:
        return VERDICT_REJECTED, blocking[0]["message"]

    warnings = [c for c in checks if not c["passed"] and c["severity"] == "warning"]
    soft = [c for c in checks if c["passed"] and c["severity"] == "warning"]

    if confidence is None:
        return VERDICT_REVIEW, "No classifier available; manual review required."

    if confidence < AUTO_REJECT_CONFIDENCE:
        return VERDICT_REVIEW, (
            f"Document type is unclear (confidence {confidence:.0%}); manual review required."
        )

    if warnings:
        return VERDICT_REVIEW, warnings[0]["message"]

    if confidence < AUTO_APPROVE_CONFIDENCE:
        return VERDICT_REVIEW, (
            f"Checks passed but classification confidence is only {confidence:.0%}."
        )

    if soft:
        return VERDICT_REVIEW, soft[0]["message"]

    return VERDICT_APPROVED, "All automated checks passed."
```

The policy is deliberately asymmetric. A blocking check rejects outright. Confidence below 0.60 goes to a human. Any warning goes to a human. Only a confident classification with entirely clean checks approves automatically.

The reasoning is that the two errors do not cost the same. Wrongly approving a forged or expired document creates a compliance problem. Wrongly sending a valid document for review costs somebody thirty seconds.

---

8. Retrieval

8.1 Chunking

Documents are split on markdown section headings. Each chunk has its document title prepended, so that a chunk retrieves on its subject even when the section text never repeats it.

```
def _chunk(document: dict) -> list[dict]:
    """Split on markdown sections, keeping each chunk self-describing."""
    body = document["body"]
    parts = re.split(r"\n(?=##\s)", body)
    chunks = []
    for part in parts:
        text = part.strip()
        if len(text) < 40:
            continue
        heading = text.split("\n", 1)[0].lstrip("# ").strip()
        chunks.append({
            "doc_id": document["id"],
            "doc_title": document["title"],
            "section": heading,
            "org": document["org"],
            "departments": document["departments"],
            # The title is prepended so a chunk retrieves on its document's
            # subject even when the section text never repeats it.
            "text": f"{document['title']} — {heading}\n{text}",
            "raw": text,
        })
    return chunks
```

8.2 Hybrid scoring

Scoring combines a lexical and a semantic component in equal measure.

```
def search(query: str, org_id: str | None = None, department: str | None = None,
           k: int = 4) -> list[dict]:
    index = build_index()
    if not index.get("chunks") or not query or not query.strip():
        return []

    query_tfidf = normalize(index["vectorizer"].transform([query]))
    lexical = (index["tfidf"] @ query_tfidf.T).toarray().ravel()

    query_latent = normalize(index["svd"].transform(query_tfidf))
    semantic = (index["latent"] @ query_latent.T).ravel()

    combined = LEXICAL_WEIGHT * lexical + (1 - LEXICAL_WEIGHT) * np.clip(semantic, 0, None)

    results = []
    for position in np.argsort(combined)[::-1]:
        chunk = index["chunks"][position]
        if not _visible(chunk, org_id, department):
            continue
        score = float(combined[position])
        if score < RELEVANCE_FLOOR:
            break
        results.append({
            "doc_id": chunk["doc_id"],
            "doc_title": chunk["doc_title"],
            "section": chunk["section"],
            "text": chunk["raw"],
            "score": score,
            "lexical": float(lexical[position]),
            "semantic": float(semantic[position]),
        })
        if len(results) >= k:
            break
    return results
```

The lexical half is cosine similarity over the TF-IDF matrix, which catches exact policy wording such as lockout-tagout. The semantic half is cosine similarity over a `TruncatedSVD` projection of the same matrix, which is latent semantic analysis.

LSA finds the directions of greatest variance in the term document matrix. Terms that co-occur across documents collapse onto nearby directions, so a query using different words from the policy text can still match. A measured example from this corpus: the query about taking time off before probation ends scores 0.339 lexically against the relevant section but 0.846 semantically.

LSA was chosen over a neural embedding model because the system had to run offline with no model download.

8.3 Access control

A chunk is visible only if its organisation matches, and where the document restricts a department, only to that department. This is applied during retrieval rather than after, so a restricted passage never enters the candidate set.

```
def _visible(chunk, org_id, department) -> bool:
    if chunk["org"] not in ("all", org_id):
        return False
    if chunk["departments"] and department not in chunk["departments"]:
        return False
    return True
```

8.4 Gap detection

Two thresholds. Below `RELEVANCE_FLOOR` nothing is returned at all. Between that and `WEAK_MATCH_SCORE` the passages are returned but flagged to the model as possibly irrelevant, and the question is recorded as a documentation gap.

The second threshold was added after testing showed that the common failure is not an empty result but a topically adjacent one. Asking about parental leave retrieves the leave policy, which is related but does not answer the question.

---

9. The Risk Model

9.1 Building features from live data

```
def build_features(db, employee) -> tuple[np.ndarray, dict]:
    tasks = employee.tasks or []
    docs = employee.requiredDocuments or []
    kt = employee.ktSections or []

    pct_tasks = (sum(1 for t in tasks if t.completed) / len(tasks)) if tasks else 0.0
    docs_pending = sum(1 for d in docs if d.status == "pending")
    docs_rejected = sum(1 for d in docs if d.status == "rejected")
    kt_pct = (sum(1 for k in kt if k.status == "completed") / len(kt) * 100) if kt else 0.0

    uploads = (
        db.query(models.DocumentUpload)
        .filter(models.DocumentUpload.empId == employee.id)
        .all()
    )
    # A resubmission is any upload beyond the first for a given requirement.
    per_requirement = {}
    for upload in uploads:
        per_requirement[upload.requiredDocId] = per_requirement.get(upload.requiredDocId, 0) + 1
    resubmissions = sum(max(0, n - 1) for n in per_requirement.values())

    events = (
        db.query(models.Event)
        .filter(models.Event.empId == employee.id)
        .order_by(models.Event.created_at.desc())
        .all()
    )
    chat_events = [e for e in events if e.event_type.startswith("chat.")]
    sentiments = [
        e.payload.get("sentiment") for e in chat_events
        if isinstance(e.payload, dict) and e.payload.get("sentiment") is not None
    ]

    days_since_activity = _days_since(events[0].created_at if events else None, default=7)
    days_since_start = _days_since(employee.startDate, default=MIN_DAYS)

    raw = {
        "days_since_start": float(np.clip(days_since_start, MIN_DAYS, MAX_DAYS)),
        "stage": float(employee.stage or 0),
        "progress": float(employee.progress or 0),
        "pct_tasks_complete": float(pct_tasks),
        "days_since_last_activity": float(np.clip(days_since_activity, 0, 30)),
        "docs_pending": float(docs_pending),
        "docs_rejected": float(docs_rejected),
        "doc_resubmissions": float(resubmissions),
        "kt_completed_pct": float(kt_pct),
        "chat_messages": float(len(chat_events)),
        "avg_sentiment": float(np.mean(sentiments)) if sentiments else 0.0,
        "seniority": float(_seniority(employee.role)),
    }
    vector = np.array([[raw[f] for f in FEATURES]], dtype=float)
    return vector, raw
```

9.2 Attribution

Explanations are exact rather than sampled.

```
def _contributions(model, vector) -> dict[str, float]:
    """Per-feature contribution to the log-odds.

    For a standardised linear model these are exactly the SHAP values, since
    coef * (x - mean) / std is the marginal contribution of each feature.
    """
    try:
        if hasattr(model, "named_steps") and "scale" in model.named_steps:
            scaler = model.named_steps["scale"]
            clf = model.named_steps["clf"]
            scaled = scaler.transform(vector)[0]
            return {f: float(c * v) for f, c, v in zip(FEATURES, clf.coef_[0], scaled)}

        import shap
        explainer = shap.TreeExplainer(model)
        values = explainer.shap_values(vector)
        row = values[0] if not isinstance(values, list) else values[1][0]
        return {f: float(v) for f, v in zip(FEATURES, np.ravel(row))}
    except Exception:
        logger.exception("Risk explanation failed")
        return {}
```

For a standardised linear model, the contribution of a feature to the log odds is the coefficient multiplied by the scaled value. These are the SHAP values for that model class, so no sampling is required and the explanation is deterministic. Two calls with the same input produce identical attributions, which matters when the output is shown to an HR user who may act on it.

9.3 The train and serve defect

This module contained the most serious fault found during the project.

Two features, `response_latency_hours` and `is_remote`, were hardcoded to fixed values at prediction time while the model had been trained on values that varied across the dataset. Two learned coefficients were therefore being applied to information the model never actually received. This is train and serve skew, and no test in existence at the time would have caught it, because every individual component worked correctly.

Neither value was recoverable from the data model. There is no remote working field, and employee reply timings are not recorded. Both features were removed from the feature set and the model retrained. The reported score barely moved, which is the point. It now describes what the model genuinely observes.

---

10. The Agent Layer

10.1 Capability boundaries

Six specialists, each with its own prompt and its own tool subset. The registry is the single place a capability is granted.

```
SPECIALISTS = {
    "Onboarding Agent": {
        "tools": ["get_my_status", "list_my_tasks", "complete_task"],
        "remit": (
            "You handle onboarding progress: what stage the employee is at, what "
            "tasks remain, and marking tasks complete. You are also the default "
            "for general conversation."),
    },
    "Document Agent": {
        "tools": ["list_my_documents", "list_my_tasks"],
        "remit": (
            "You handle required documents: which are outstanding, which were "
            "rejected and why, and what the employee must do next. You cannot "
            "upload files for them; explain that they upload from their dashboard."),
```

Boundaries are enforced by construction rather than by instruction. The Policy agent is never handed the tool that completes a task, so no prompt can persuade it to complete one.

10.2 Tools as an authorisation boundary

Every tool is built inside a closure over the authenticated employee identifier, which is never a model supplied argument.

```
def _t_list_my_documents(ctx: Context):
    @tool
    def list_my_documents() -> str:
        """List required documents, their verification status and any rejection reason."""
        db = SessionLocal()
        try:
            docs = (db.query(models.RequiredDocument)
                    .filter(models.RequiredDocument.empId == ctx.emp_id).all())
            if not docs:
                return "No documents required."
            lines = []
            for d in docs:
                line = f"- {d.name}: {d.status}"
                if d.reason:
                    line += f" (reason: {d.reason})"
                lines.append(line)
            return "\n".join(lines)
        finally:
            db.close()
    return list_my_documents
```

The identifier comes from the request, not from the model. A prompt instructing the agent to look up a different employee cannot succeed, because there is no parameter through which another identifier could be passed. This was tested with a direct injection attempt and held.

An important caveat for anybody extending this: the property holds for the chat path only. The REST API beneath it has no authentication at all, so the same data is reachable by calling the endpoints directly.

10.3 Routing

```
def route(message: str) -> dict:
    """Pick a specialist for a message using the intent classifier."""
    analysis = nlp_models.analyse(message)
    agent = analysis.get("agent") or specialists.DEFAULT_SPECIALIST
    if agent not in specialists.SPECIALISTS:
        agent = specialists.DEFAULT_SPECIALIST
    return {"specialist": agent,
            "intent": analysis.get("intent"),
            "confidence": analysis.get("intent_confidence"),
            "routed": analysis.get("routed", False)}
```

Routing uses the trained intent classifier rather than asking a language model which agent to use. This is one model call cheaper, it is deterministic and therefore testable, and its accuracy can be measured against a confusion matrix.

Below the confidence floor nothing is routed and the default specialist answers, because a vague message genuinely does not identify a specialist.

A specialist may transfer once if a message reaches the wrong one, which covers the classifier's residual error without risking a loop.

10.4 Content blocks

```
def _as_text(content) -> str:
    """Flatten a message's content to plain text.

    Newer models (Gemini 3.x among them) return a list of typed blocks, some of
    which are internal reasoning that must not be shown to the employee.
    """
    if isinstance(content, str):
        return content.strip()
    if isinstance(content, list):
        parts = []
        for block in content:
            if isinstance(block, str):
                parts.append(block)
            elif isinstance(block, dict) and block.get("type") == "text":
                parts.append(block.get("text", ""))
        return "\n".join(p for p in parts if p).strip()
    return str(content).strip() if content else ""
```

Newer models return message content as a list of typed blocks rather than a string, and some of those blocks are internal reasoning. Converting that list to a string naively would have shown the model's reasoning to the employee.

---

11. The Web Layer

11.1 Schema

```
organizations      id, name, industry, logo
employees          id, name, role, department, orgId, startDate, status, stage, progress
required_documents id, name, status, reason, empId
kt_sections        id, name, status, contact, empId
tasks              id, title, completed, empId
audit_logs         log_id, time, action, empId          prose for humans
events             id, empId, event_type, actor, payload, confidence, created_at
document_uploads   id, empId, requiredDocId, stored_path, sha256,
                   ocr_text, predicted_type, predicted_confidence,
                   extracted_fields, validations, verdict, model_version,
                   hr_decision, hr_decided_at
```

Two columns on `document_uploads` are kept deliberately apart. `verdict` is what the model concluded and `hr_decision` is what a human concluded. The disagreement between them is the training signal for improving the classifier, and it is lost if one overwrites the other.

11.2 Request lifecycle for a chat message

```
POST /api/chat
  |
  1. nlp_models.analyse(message)        intent, confidence, sentiment score
  2. pii.redact(message)                strip identifiers before anything leaves
  3. escalation check                   intent or sentiment below threshold
  4. process_chat_message(safe, emp_id, specialist)
       route to specialist, build tools in a closure, invoke, allow one transfer
  5. events.record(chat.message)        sentiment lands in the feature store
  6. response with agent, intent, confidence, redaction list
```

11.3 Upload safety

Three controls, each tested.

```
def sniff_type(head: bytes, declared: str | None) -> str:
    """Identify the file from its leading bytes, ignoring the declared type."""
    for mime, (_, signatures) in ALLOWED_TYPES.items():
        if any(head.startswith(sig) for sig in signatures):
            return mime
    raise UploadRejected(
        "Unsupported file type. Only PDF, JPEG and PNG files are accepted."
    )
```

The content type declared by the browser is ignored entirely. The real type is identified from the leading bytes, so an HTML file labelled as a PDF is rejected.

Stored filenames are generated UUIDs, never the name supplied by the user, which removes an entire class of path manipulation problems. Resolution refuses any path that escapes the storage root.

11.4 Provider abstraction

```
def get_llm(temperature: float = 0.0, model: str | None = None):
    """Return a LangChain chat model for the configured provider.

    Imports are deferred so that an uninstalled fallback provider never breaks
    the default path.
    """
    provider = active_provider()
    model = model or active_model()

    if provider == "ollama":
        from langchain_ollama import ChatOllama
        return ChatOllama(model=model, temperature=temperature, base_url=OLLAMA_BASE_URL)

    if provider == "gemini":
        from langchain_google_genai import ChatGoogleGenerativeAI
        return ChatGoogleGenerativeAI(model=model, temperature=temperature)

    if provider == "groq":
        from langchain_groq import ChatGroq
        return ChatGroq(model=model, temperature=temperature)

    raise ValueError(
        f"Unknown LLM_PROVIDER '{provider}'. Expected one of: {', '.join(DEFAULT_MODELS)}"
    )
```

This is the only module in the project that names a model vendor. Imports are deferred so that an uninstalled fallback provider never breaks the default path.

The abstraction was not speculative. When the cloud provider returned a quota error during testing, switching to a local model required one environment variable and no code change.

---

12. Testing

34 cases expanding to 55 checks, running offline in about twenty seconds.

```
tests/conftest.py                 fixtures, including temp storage redirection
tests/test_data_quality.py        TC-01 to TC-08
tests/test_preprocessing_and_split.py  TC-09 to TC-15
tests/test_pipeline_and_safety.py  TC-16 to TC-26
tests/test_nlp_retrieval_agents.py TC-27 to TC-34
```

Two rules constrain the suite. No test calls a language model, because a result that depends on sampling temperature cannot act as a regression gate. All file tests redirect storage to a temporary directory, so running the suite never alters project state.

Two cases assert that a known defect is present and bounded, rather than absent. They fail if the defect grows worse, and they fail if somebody fixes it without updating the documentation.

---

13. Defects Found During Development

These are recorded because the root causes are instructive.

Defect	Root cause	Fix
Valid passport marked expired	Whole line keyword search assigned the first date to the expiry role when both dates shared a line	Nearest preceding label logic, regression test TC-18
Certificate number read as IFICATE	The abbreviation cert matched the start of the word CERTIFICATE in a title	Pattern requires a qualifier token, regression test TC-19
Two risk features ignored at inference	Hardcoded constants at serving time while trained on varying values	Features removed, model retrained
Positive samples with no positive signal	Both positive and neutral prefix pools contained an empty string	Marker guaranteed unless the intent carries tone
Agent error reported as a network failure	The handler blamed connectivity for every exception	Rate limit, connectivity and generic faults separated
Employee dashboard never updated	currentUser was a snapshot taken at login and never refreshed	Context re-syncs when the employee list is refetched
Database created twice	Relative SQLite path resolved against the working directory	Path anchored to the module file
Environment file not loaded	A bare load_dotenv looks in the working directory, not beside the module	Explicit path
Runaway CPU during development	The uvicorn reload watcher scanned node_modules and both virtual environments	Reload disabled for the preview server

14. Extending the System

14.1 Adding a document class

1. Add templates to `_TEMPLATES` in `ml/synth.py` and the class name to `DOC_CLASSES`.
2. Map the requirement wording to the class in `_REQUIREMENT_KEYWORDS` in `ml/validate.py`.
3. If the class needs a signature or an expiry, add it to `_SIGNATURE_REQUIRED` or

`_EXPIRY_REQUIRED`.

1. Retrain with `python -m ml.train_classifier`.
2. Regenerate the report and check the per tier accuracy has not fallen.

14.2 Adding an agent tool

1. Write a builder function in `agents/specialists.py` following the existing pattern. It

must take the context and close over `ctx.emp_id`, never accept an identifier as a model supplied argument.

1. Register it in `TOOL_BUILDERS`.
2. Add its name to the `tools` list of exactly the specialists that should hold it.
3. Add a case to TC-33 asserting which specialists do and do not have it.

14.3 Adding a risk feature

1. Generate it in `ml/risk_synth.py` and append it to `FEATURES`.
2. Compute it from live data in `build_features` in `ml/risk_model.py`. If it cannot be

computed from real data, do not add it. That is how the train and serve defect occurred.

1. Add a label to `FEATURE_LABELS`.
2. Retrain and check the ablation output to see whether it earns its place.

14.4 Adding a policy document

Drop a markdown file into `backend/knowledge/` with the metadata header, then restart. The index rebuilds lazily on first search.

```
<!-- title: Leave Policy (Nimbus Labs) -->
<!-- org: org_1 -->
<!-- departments: Engineering,Legal -->
```

Omit the departments line to make the document visible to the whole organisation.

---

15. Debugging

Symptom	Likely cause
Classifier returns a null prediction	Artefact missing. Run `python -m ml.train_classifier`
Retrieval returns nothing for every query	Corpus not seeded. Run `python -m ml.knowledge_seed`
Chat says AI features are offline	Check `GET /api/health`. It probes the provider rather than only reading an environment variable
Risk endpoint returns available false	Risk model not trained, or the feature count does not match `FEATURES`
Tests fail after changing a generator	Expected. The suite asserts dataset shape and balance. Update the assertions deliberately
Upload rejected with no obvious reason	Magic byte check. The declared content type is ignored
Port 8000 already in use	Another project may hold it. The frontend hardcodes this port in `AppContext.jsx`

15.1 Useful inspection commands

```
# what the model concluded about every upload
python -c "import models; from database import SessionLocal; d=SessionLocal(); \
  [print(u.id, u.predicted_type, u.verdict) for u in d.query(models.DocumentUpload).all()]"

# the event stream for one employee
curl http://localhost:8000/api/employees/emp_1/events

# classifier agreement with HR so far
curl http://localhost:8000/api/ml/agreement
```

---

16. Known Limitations

1. All datasets are synthetic. Every figure measures consistency with the generators, not real world capability.
1. Sentiment accuracy is a template separability ceiling and must not be read as model capability.

1. The message corpus contains duplicates that straddle the split.
2. Three journey features correlate above 0.90, so the feature set is redundant.
3. The REST API has no authentication. The agent layer's scoping does not protect it.
4. There are no API level or frontend tests.
5. The project is not under version control.